Data Dictionary for `adult.csv`

Column Name	Data Type	Description	Example Values
age	Integer	Age of the individual (continuous variable).	25, 38, 28, 44, 18
workclass	Categorical	Type of employment classification.	Private, Local-gov, Self-emp-not-inc, Federal-gov, ?
fnlwgt	Integer	Final weight, number of people the census believes the entry represents.	226802, 89814, 336951, 160323, 103497
education	Categorical	Highest level of education achieved.	11th, HS-grad, Assoc-acdm, Some-college, 10th
educational-num	Integer	Numeric representation of education level.	7, 9, 12, 10, 6
marital-status	Categorical	Marital status of the individual.	Never-married, Married-civ-spouse, Widowed, Divorced
occupation	Categorical	Type of occupation.	Machine-op-inspct, Farming-fishing, Protective-serv
relationship	Categorical	Family relationship within a household.	Own-child, Husband, Not-in-family, Unmarried
race	Categorical	Race of the individual.	Black, White, Asian-Pac-Islander, Other
gender	Categorical	Gender of the individual.	Male, Female
capital-gain	Integer	Capital gains from investments.	0, 7688, 3103, 6418, 7298
capital-loss	Integer	Capital losses from investments.	0, 1721, 1876, 2415, 1887
hours-per-week	Integer	Average number of work hours per week.	40, 50, 30, 32, 10

native-country	Categorical	Native country of the individual.	United-States, Peru, Guatemala, Mexico, ?
income	Categorical	Income category of the individual.	<=50K, >50K

Here’s the finalized document content in a clean, copy-ready format:

Working with the Dataset – Guidelines & Best Practices

1. Important Notes

Missing Values

- `"?"` denotes missing values in **workclass**, **occupation**, and **native-country**.
- Replace with `NaN` before encoding.

Labels

- Some files contain dotted variants: `>50K.`, `<=50K.`.
- Normalize labels before mapping to binary.

Skewed Features

- **capital_gain**, **capital_loss**, and **fnlwgt** are heavily skewed.
- Consider transformations (`log1p`) or binary indicators.

Sensitive Attributes

- Columns such as **sex**, **race**, and **marital-status** require **fairness audits**.

- Decisions must be documented for compliance and transparency.

---

2. Common Transformations & Feature Engineering

Indicator Variables

- `capital_gain_gt0 = 1 if capital_gain > 0 else 0`
- `capital_loss_gt0 = 1 if capital_loss > 0 else 0`

Binning

- `age_bin = (18–25, 26–35, 36–45, 46–55, 56+)`
- `hours_bin = (≤30, 31–40, 41–50, 51+)`

Interaction Features

- `age_x_education = age × education-num`

Encoding Guidance

- **One-hot encoding:** sex, race, relationship
- **Target/Frequency/CatBoost encoding:** occupation, native-country

---

3. Data Quality & Validation Rules

Type Enforcement

- **Numeric:** age, fnlwgt, education-num, capital-gain, capital-loss, hours-per-week

Range Checks

- `16 ≤ age ≤ 100`
- `0 ≤ hours-per-week ≤ 100`
- `capital-gain ≥ 0`
- `capital-loss ≥ 0`

Handling Missing Values

- `"?" → NaN`
- Impute categorical variables with **most frequent** value or `"Unknown"`.
- Impute numeric variables with **median**.

---

4. Getting Started – Loading & Normalizing Data

```
import numpy as np, pandas as pd


# Load dataset
df = pd.read_csv('adult.csv')


# Clean column names
df.columns = [c.strip().lower().replace('-', '_') for c in df.columns]


# Normalize labels
df['income'] = df['income'].astype(str).str.strip().map({
    '>50K': 1, '<=50K': 0,
```

```
    '>50K.': 1, '<=50K.': 0
})

# Replace '?' with NaN
df = df.replace('?', np.nan)
```

---

5. Colab Usage – FeatureBot Automated Pipeline

Instructions

1. In **Colab**, click **File > New notebook**.
2. Upload `adult.csv` to the working directory (or adjust `DATA_PATH`).
3. Create a new file named **featurebot_adult_income.py**, paste the script from *Appendix A*, and save.

Run in a notebook cell:

```
%run featurebot_adult_income.py
```

4.

Optionally, import the module:

```
import featurebot_adult_income as fb
```

5.

Appendix A – FeatureBot Script (Full)

```
# featurebot_adult_income.py
# Project: FeatureBot - AI-Guided Feature Engineering for UCI Adult Income
```

```
# Tools: Google Colab, Python (pandas, scikit-learn)

# Skills: Feature Engineering, Prompting (ChatGPT used outside this script)




# Install libraries if needed (Colab usually has these pre-installed)
!pip install category_encoders

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, precision_recall_fscore_support, classification_report
import matplotlib.pyplot as plt
import category_encoders as ce


df = pd.read_csv('/content/adult.csv', sep=',', on_bad_lines='skip')
display(df.head())



#df = pd.read_csv('/content/adult.csv')
df.columns = [c.strip().lower().replace("-", "_") for c in df.columns]
df = df.replace('?', np.nan)  # "?" to nan for missing
if 'income' in df:
    df['income'] = df['income'].astype(str).str.strip().replace({'>50K.': '>50K', '<=50K.': '<=50K'})
    df['income'] = df['income'].map({'>50K': 1, '<=50K': 0})




print(df.head())
print("\nMissing Counts:\n", df.isnull().sum())
print("\nClass Balance:\n", df['income'].value_counts(normalize=True))
df.describe(include='all').T
```

```
# Numeric distributions
df.select_dtypes(include='number').hist(figsize=(15,8), bins=30)
plt.suptitle("Numeric Distributions")
plt.show()

# Value counts for categoricals
for col in df.select_dtypes('object').columns:
    print(f"\n{col} value counts:\n", df[col].value_counts(dropna=False)[:10])
```

```
numeric_tf = Pipeline([
    ('imp', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
categorical_tf = Pipeline([
    ('imp', SimpleImputer(strategy='most_frequent')),
    ('ohe', OneHotEncoder(handle_unknown='ignore'))
])
preprocess = ColumnTransformer([
    ('num', numeric_tf, num_cols),
    ('cat', categorical_tf, cat_cols)
])
baseline = Pipeline([
    ('prep', preprocess),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
]).fit(X_train, y_train)

def report_metrics(y_true, y_pred, y_prob):
    prec, rec, f1, _ = precision_recall_fscore_support(y_true, y_pred, average='binary', zero_division=0)
    auc = roc_auc_score(y_true, y_prob)
    print(f"AUC: {auc:.3f} | F1: {f1:.3f} | Precision: {prec:.3f} | Recall: {rec:.3f}")
    return {'AUC': auc, 'F1': f1, 'Precision': prec, 'Recall': rec}

yval_pred = baseline.predict(X_val)
yval_prob = baseline.predict_proba(X_val)[:,1]
baseline_metrics = report_metrics(y_val, yval_pred, yval_prob)
```

```
# Add features as per your function from last step, e.g.:
def add_engineered_features(df):
    df = df.copy()
    df["age_bin"] = pd.cut(df["age"], bins=[0, 25, 35, 45, 55, 65, 100], labels=["<25", "25-35", "35-45", "45-55",
"55-65", "65+"])
    df["capital_net"] = df["capital_gain"].fillna(0) - df["capital_loss"].fillna(0)
    df["has_capital_gain"] = (df["capital_gain"].fillna(0) > 0).astype(int)
    df["hours_bin"] = pd.cut(df["hours_per_week"], bins=[0, 29, 39, 49, 59, 100], labels=["<30", "30-39", "40-49",
"50-59", "60+"])

    if "marital_status" in df.columns and "gender" in df.columns:
        df["marital_gender"] = df["marital_status"].astype(str) + "_" + df["gender"].astype(str)
    # Safe target encoding (DO NOT leak! This is safe if done only on train)
    if "occupation" in df.columns and "income" in df.columns: # Corrected column check
        occupation_te = df.groupby("occupation")["income"].transform("mean")
        df["occupation_te"] = occupation_te
    return df

# Apply to all splits
X_train_e = add_engineered_features(X_train)
X_val_e = add_engineered_features(X_val)
X_test_e = add_engineered_features(X_test)

# Get new col types
num_cols_e = X_train_e.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols_e = X_train_e.select_dtypes(include=['object', 'category']).columns.tolist()

# Preprocessing and model pipeline
preprocess_e = ColumnTransformer([
    ('num', SimpleImputer(strategy='median'), num_cols_e),
    ('cat', Pipeline([
        ('imp', SimpleImputer(strategy='most_frequent')),
        ('ohe', OneHotEncoder(handle_unknown='ignore'))
    ]), cat_cols_e)
])
pipe_e = Pipeline([
    ('prep', preprocess_e),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
]).fit(X_train_e, y_train)

# Evaluate on val
yval_pred_e = pipe_e.predict(X_val_e)
yval_prob_e = pipe_e.predict_proba(X_val_e)[:,1]
def report_metrics(y_true, y_pred, y_prob):
```

```
    prec, rec, f1, _ = precision_recall_fscore_support(y_true, y_pred, average='binary', zero_division=0)
    auc = roc_auc_score(y_true, y_prob)
    print(f"AUC: {auc:.3f} | F1: {f1:.3f} | Precision: {prec:.3f} | Recall: {rec:.3f}")
    return {'AUC': auc, 'F1': f1, 'Precision': prec, 'Recall': rec}
enhanced_metrics = report_metrics(y_val, yval_pred_e, yval_prob_e)
print(classification_report(y_val, yval_pred_e))
```

```
num_cols_e = X_train_e.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols_e = X_train_e.select_dtypes(include=['object','category']).columns.tolist()
preprocess_e = ColumnTransformer([
    ('num', numeric_tf, num_cols_e),
    ('cat', categorical_tf, cat_cols_e)
])
enhanced = Pipeline([
    ('prep', preprocess_e),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
]).fit(X_train_e, y_train)
yval_pred_e = enhanced.predict(X_val_e)
yval_prob_e = enhanced.predict_proba(X_val_e)[:,1]
enhanced_metrics = report_metrics(y_val, yval_pred_e, yval_prob_e)
```

```
def fairness_by_group(X, y_true, y_pred, y_prob, group):
    records = []
    for g in X[group].dropna().unique():
        idx = X[group] == g
        auc = roc_auc_score(y_true[idx], y_prob[idx])
        prec, rec, f1, _ = precision_recall_fscore_support(y_true[idx], y_pred[idx], average='binary',
zero_division=0)
        records.append({'group':g, 'AUC':auc, 'F1':f1, 'Prec':prec, 'Recall':rec})
    return pd.DataFrame(records)

print(fairness_by_group(X_val_e, y_val, yval_pred_e, yval_prob_e, "gender"))
print(fairness_by_group(X_val_e, y_val, yval_pred_e, yval_prob_e, "race"))
```

```
# Combine train & val for final fit
X_final = pd.concat([X_train_e, X_val_e])
y_final = pd.concat([y_train, y_val])
final_model = Pipeline([
    ('prep', preprocess_e),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
]).fit(X_final, y_final)
ytest_pred = final_model.predict(X_test_e)
ytest_prob = final_model.predict_proba(X_test_e)[:,1]
report_metrics(y_test, ytest_pred, ytest_prob)
print(classification_report(y_test, ytest_pred))
```